

APACHE SPARK

FUNDAMENTALS



By Michael Fernandes

TABLE OF CONTENTS

1

Introduction to Spark

- Limitations of Hadoop MapReduce
- What is Apache Spark
- Why Apache Spark
- Features of Spark
- Hadoop vs Spark

2

Spark Architecture

- Spark Ecosystem Overview
- Spark Architecture
- Spark Execution Workflow

3

RDD Fundamentals

- What is RDD
- RDD Operations
- Lazy Evaluation
- Caching and Persistence

4

DataFrames and Spark SQL

- What are DataFrames
- Common DataFrame Operations
- Spark SQL
- Catalyst Optimizer

5

Spark Processing Concepts

- DAG (Directed Acyclic Graph)
- Narrow vs Wide Transformations
- Shuffle Operations
- Partitioning

6

Spark Streaming

- What is Spark Streaming
- Streaming Workflow
- Use Cases

7

MLlib Fundamentals

- What is MLlib
- ML Pipeline Workflow
- Common Algorithms

8

Spark Use Cases

9

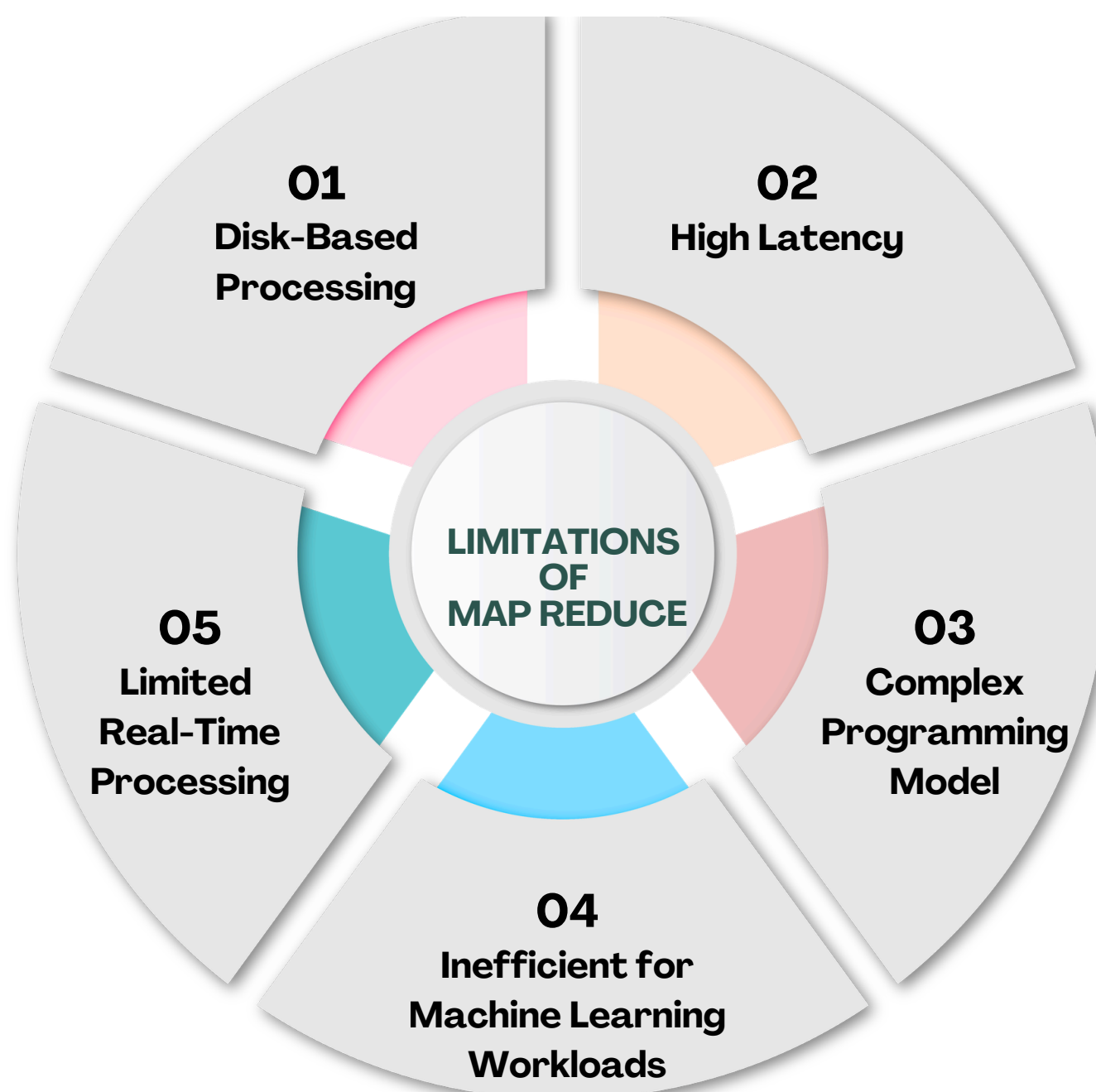
Spark Installation and Ecosystem Integration

Spark + Hadoop
Spark + Hive
Spark + Kafka
Spark + Airflow

Part I - Introduction to Spark

1.1

Limitations Of MapReduce



1. Disk-Based Processing

MapReduce writes intermediate results to disk after every processing stage.

Why This Is a Problem

- Heavy disk I/O operations
- Increased execution time
- Slower data processing pipelines

Example - A multi-stage analytics job repeatedly:

1. Reads data from disk
2. Processes data
3. Writes output back to disk

This creates major performance bottlenecks.

2. High Latency

MapReduce jobs have high execution latency because:

- Every stage depends on disk operations
- Job startup overhead is large
- Intermediate data transfer is slow

Result

MapReduce is inefficient for:

- Real-time analytics
- Interactive querying
- Streaming applications

It is mainly suitable for long-running batch workloads.

3. Complex Programming Model

Developers must manually write:

- Mapper logic
- Reducer logic
- Key-value transformations
- Job configurations

Challenges

- Large amount of boilerplate code
- Difficult debugging
- Longer development cycles

Building complex pipelines becomes time-consuming.

4. Inefficient for Machine Learning Workloads

Machine Learning algorithms often require:

- Multiple passes over data
- Fast memory access
- Iterative optimization

MapReduce performs poorly because it lacks efficient in-memory computation.

5. Limited Real-Time Processing

MapReduce was designed primarily for batch processing.

Limitations:

It cannot efficiently support:

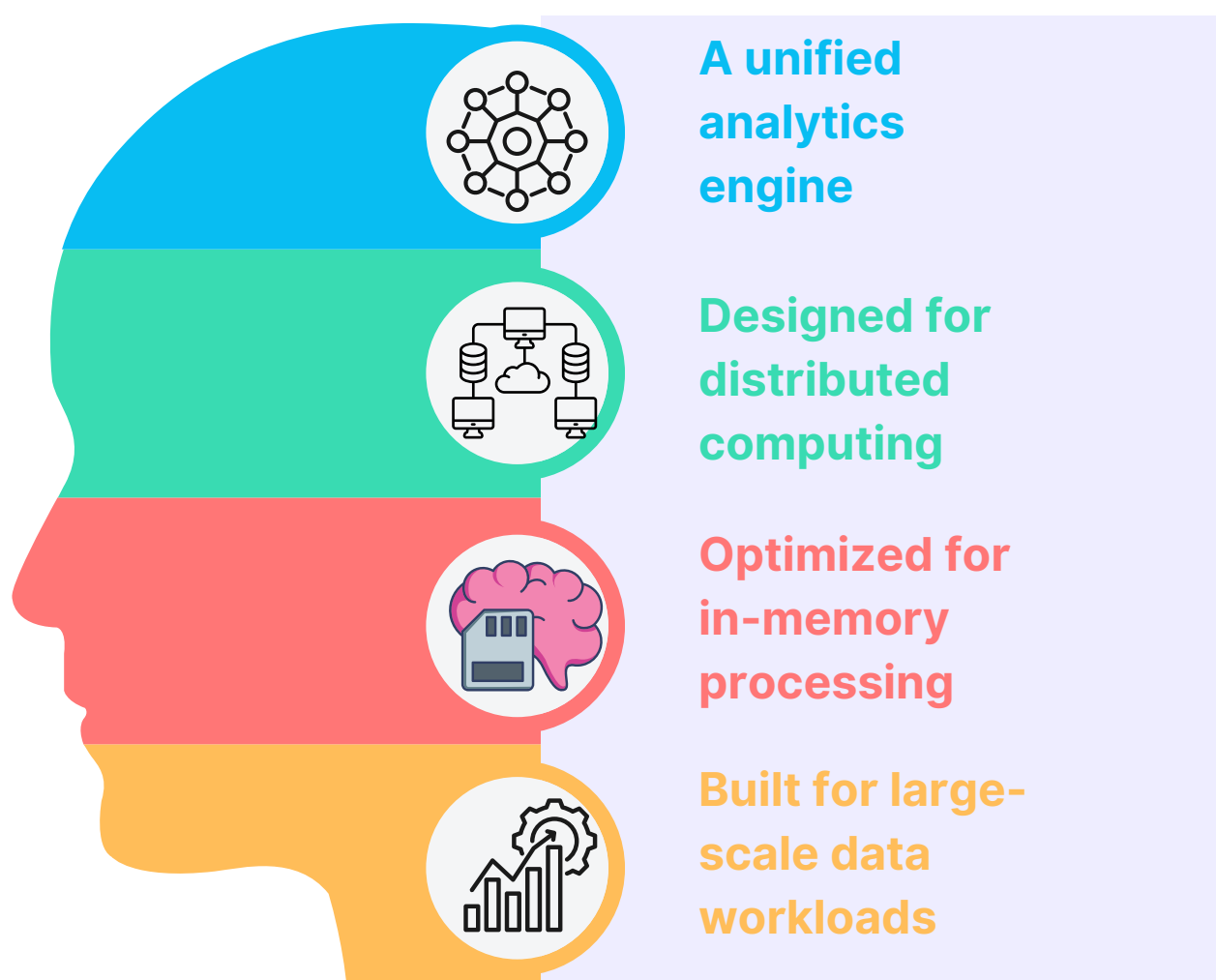
- Continuous streaming
- Live dashboards
- Event-driven systems

This created demand for modern real-time processing engines like Spark.

1.2

What is Apache Spark

Apache Spark is



```
# Creating Spark Session

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("App") \
    .getOrCreate()
```


1.3

Why Apache Spark

8.Integration with Big Data Ecosystem

Spark integrates with:Hadoop,Hive,Kafka,Delta Lake,Airflow and Cloud platforms

1.In-Memory Computing

One of Spark’s biggest advantages is its ability to process data in memory instead of repeatedly writing intermediate results to disk.

2. Distributed Processing

Spark distributes data and computations across multiple machines in a cluster.

3.Lazy Evaluation

Spark builds an execution plan and runs computations only when an action is triggered.

4. Fault Tolerance

Spark Recovers from failiures using DAG Lineage and recomputation

5. Scalability

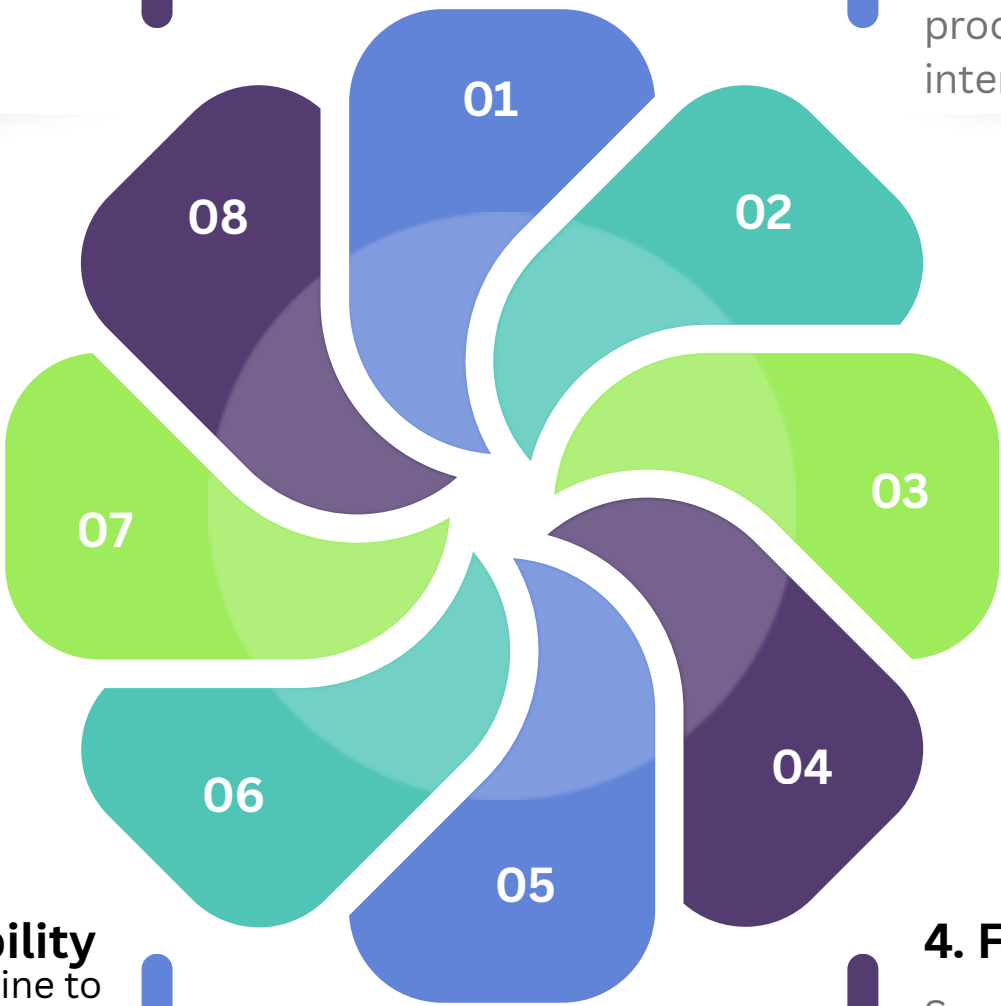
Spark can Scale from a single machine to thousands of cluster nodes

7. Unified Analytics Engine

Spark Supports SparkSQL, MLib, Graphx iand Streaming in single framework

6. Real-Time Stream Processing

Using Structured Streaming, Spark can process continuous streams of data in real time.

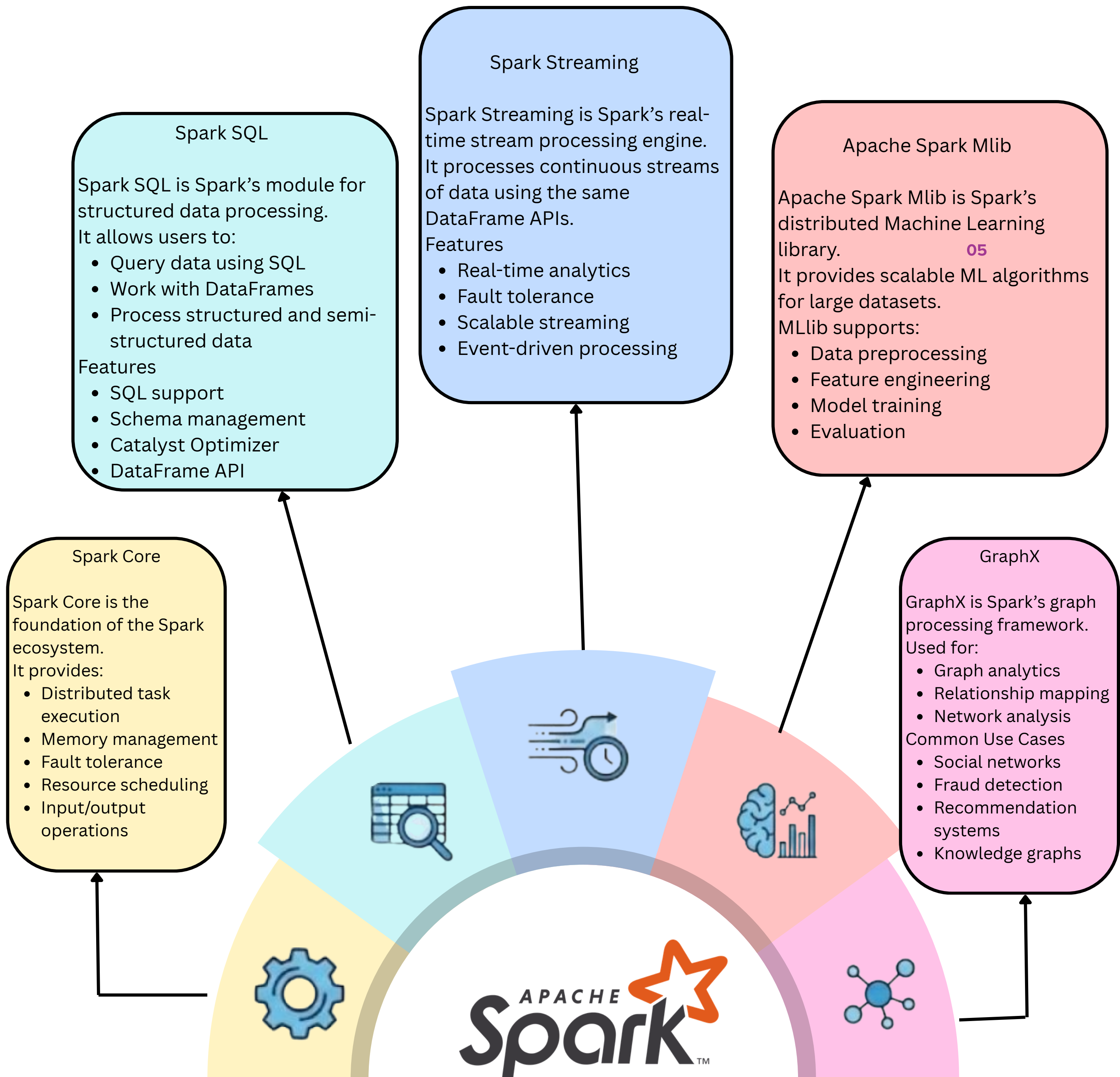


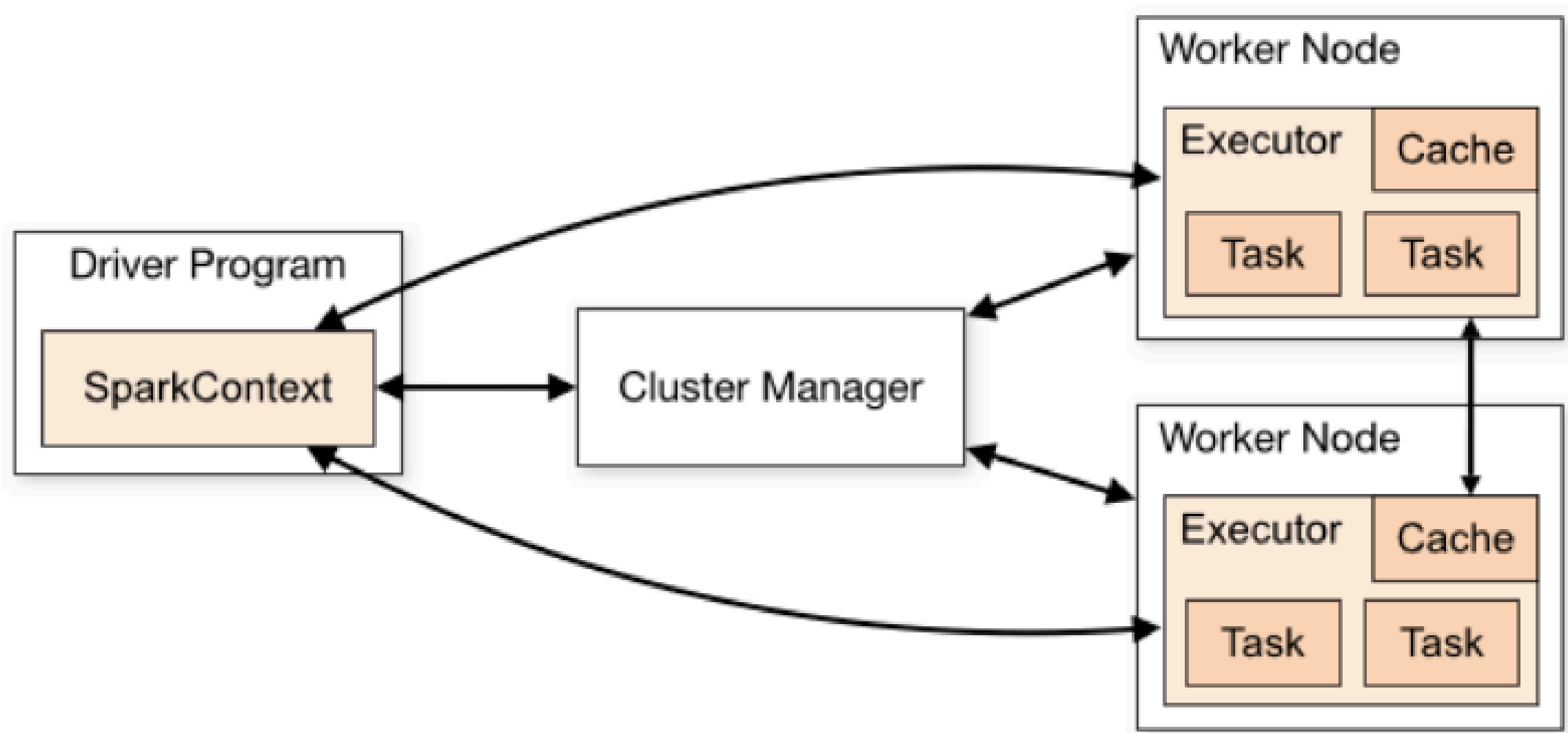
Hadoop	Spark
Primarily a storage + processing ecosystem	Primarily a fast processing engine
Uses disk-based processing	Uses in-memory processing
Slower execution	Faster execution
Disk-based	In-memory
Uses MapReduce For Execution	Uses DAG For Execution
Limited Streaming and ML	Strong Streaming and ML
Language- Java	Languages- Python, Scala, Java

Part II - Spark Architecture

2.1

Spark Ecosystem Overview





Apache Spark uses a distributed master-worker architecture to process large-scale data efficiently across multiple machines.

The architecture is designed to provide:

- Parallel processing
- Fault tolerance
- Scalability
- High-speed computation

1. Driver Program

The Driver Program is the brain of the Spark application.

Responsibilities

- Creates SparkContext
- Builds execution plans
- Converts transformations into DAG
- Schedules tasks
- Coordinates executors
- Collects final results

SparkContext

SparkContext acts as the connection between:

- Driver Program
- Spark Cluster

It is responsible for:

- Resource negotiation
- Task scheduling
- Cluster communication

2. Cluster Manager

The Cluster Manager manages cluster resources.

Responsibilities

- Allocates CPU and memory
- Launches executors
- Monitors cluster resources

Supported Cluster Managers

Standalone - Spark's built-in manager
YARN - Hadoop resource manager
Kubernetes - Container orchestration
Mesos - Distributed cluster manager

3. Worker Node

Worker nodes are machines in the cluster that perform actual computations.

Each worker node contains:

- Executors
- Cached data
- Running tasks

Executors

Executors are JVM processes launched on worker nodes.

Responsibilities

- Execute tasks
- Store cached data
- Return results to driver

Each application gets its own executors.

Tasks

A task is the smallest unit of execution in Spark.

One partition generally corresponds to one task.

Tasks execute computations in parallel across worker nodes

Cache

Spark can cache datasets in memory for faster repeated access.

Why Important

Caching improves:

- Iterative processing
- Machine Learning workloads
- Interactive analytics

Driver Program Starts

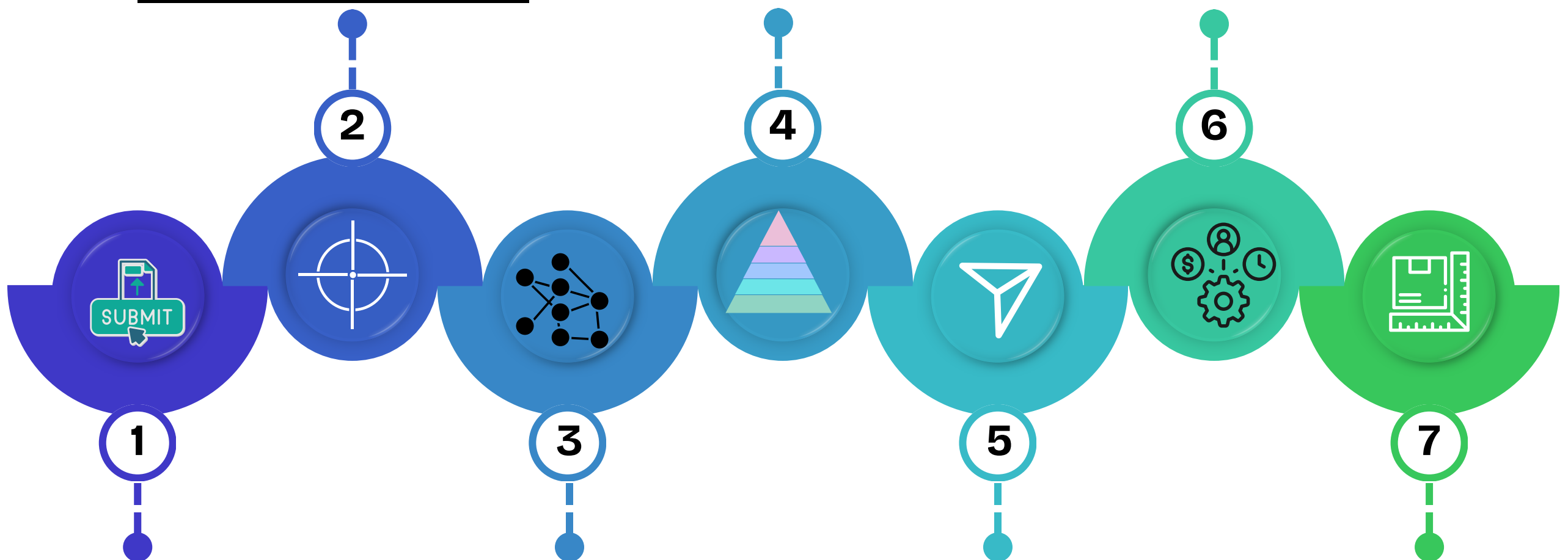
The Driver Program acts as the brain of the Spark application. It creates the SparkSession and SparkContext required for execution. The driver analyzes the user code and prepares the execution workflow. It also communicates with the Cluster Manager to request resources.

DAG Split into Stages

The DAG Scheduler divides the DAG into multiple stages. Stages are created based on shuffle boundaries in the workflow. Operations like join() and groupBy() usually create separate stages. Each stage contains multiple tasks that can run in parallel.

Cluster Manager Allocates Resources

The Cluster Manager allocates CPU and memory resources for the application. It launches executors on available worker nodes in the cluster. Spark can work with cluster managers like YARN, Kubernetes, or Standalone mode. This step ensures efficient resource utilization across the cluster.



User Submits Spark Application

The application may be written in Python, Scala, Java, or R. This application contains transformations and actions that Spark must execute. After submission, the Driver Program starts running.

DAG Creation

Spark converts all transformations into a DAG (Directed Acyclic Graph). The DAG represents the logical execution plan of the application. Spark uses lazy evaluation, so transformations are not executed immediately. Instead, Spark optimizes the DAG before actual execution begins.

Tasks Are Created

Each stage is divided into smaller tasks for parallel execution. Typically, one partition corresponds to one task. Tasks are the smallest units of execution in Spark. These tasks are distributed across executors on worker nodes.

Executors Execute Tasks

Executors run the assigned tasks in parallel on worker nodes. They process partitions of data and perform computations. Executors can also cache intermediate datasets in memory for faster processing. After execution, results are sent back to the Driver Program.

Part III - Introduction to Apache Hadoop

3.1

What is Hadoop

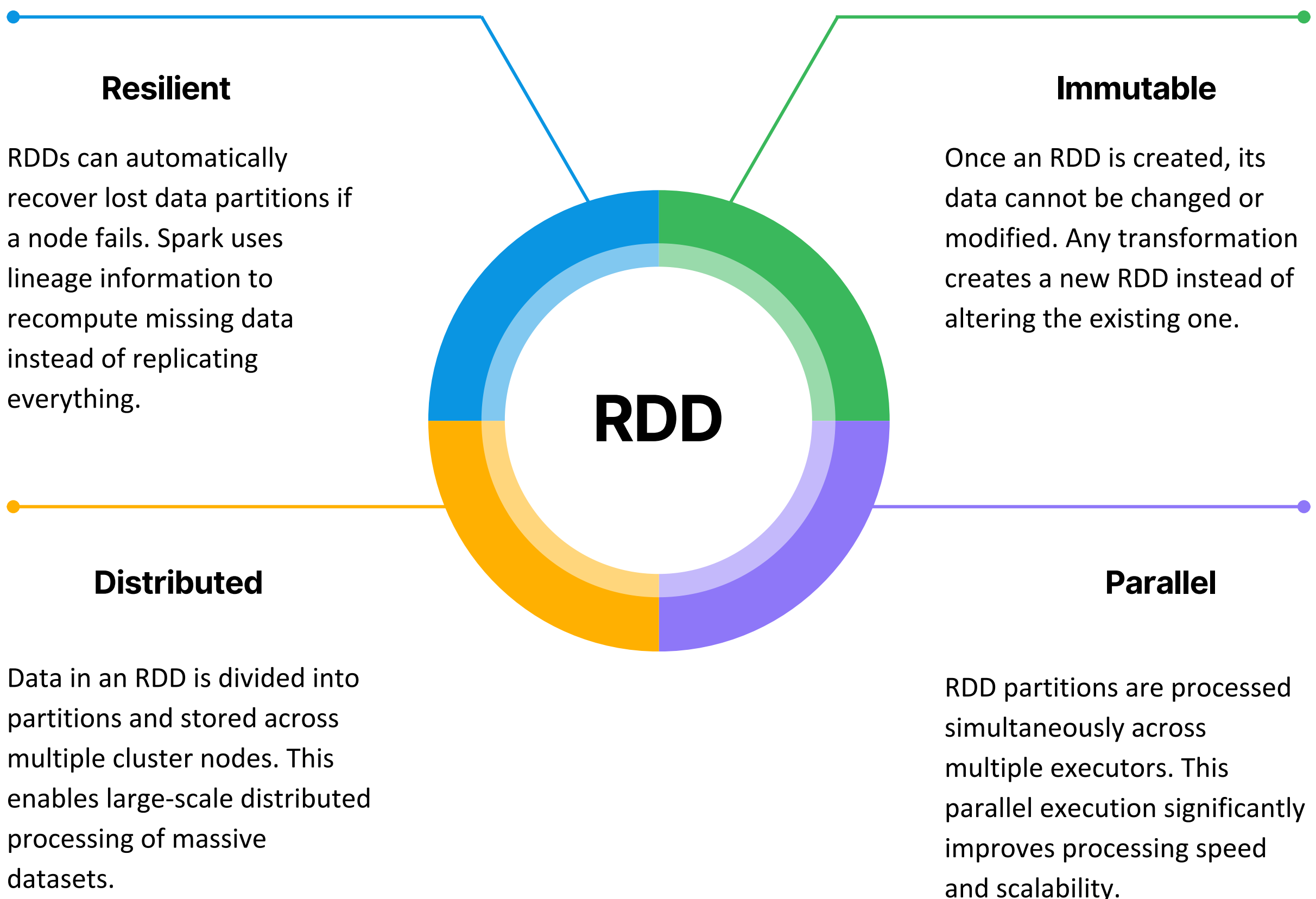
RDD stands for Resilient Distributed Dataset

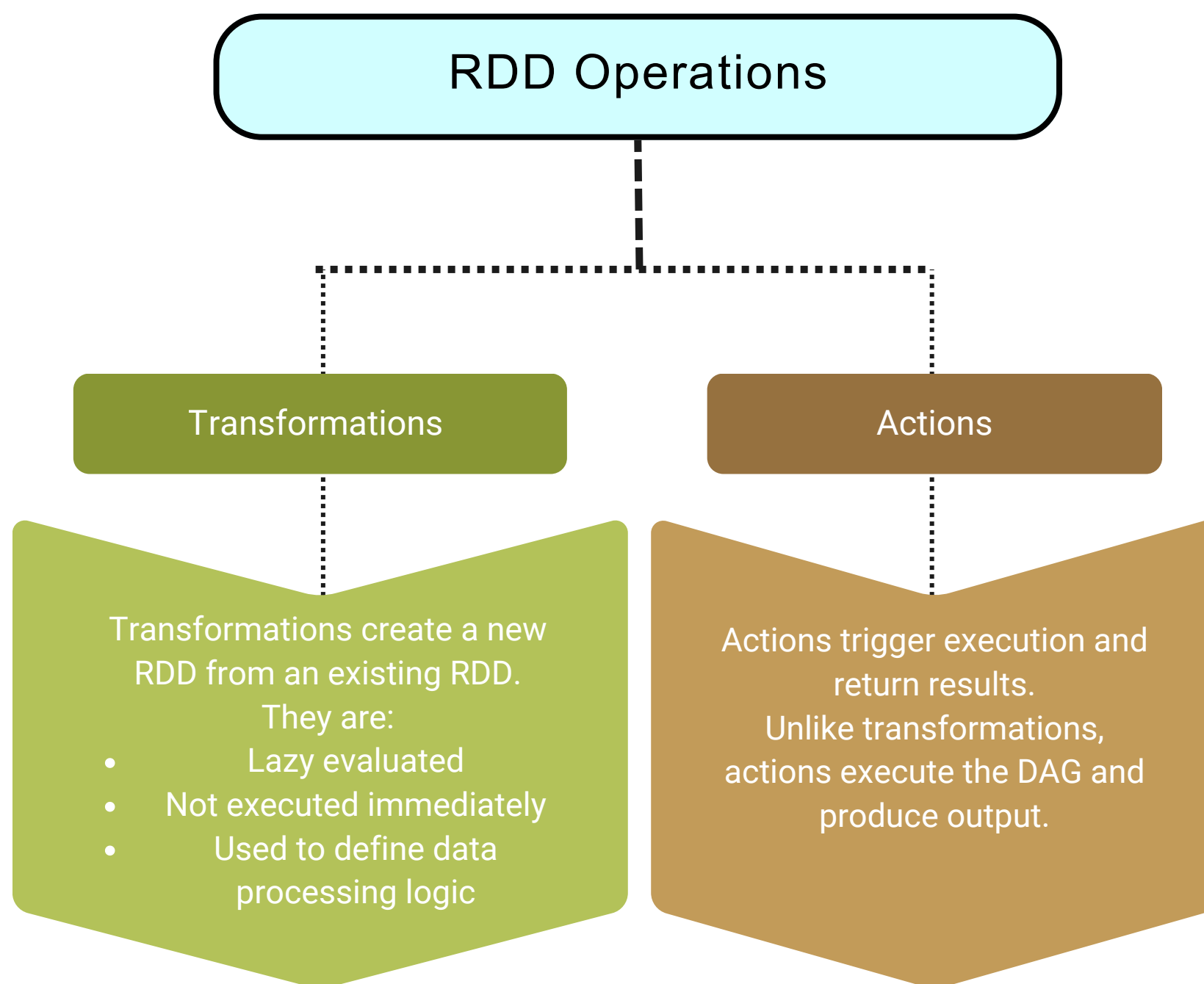
It is the fundamental data abstraction in early versions of Apache Spark.

An RDD is:

- Distributed across cluster nodes
- Immutable
- Fault tolerant
- Lazily evaluated

RDDs allow Spark to process data in parallel across multiple machines.





RDD Transformations

1. map()

Applies a function to each element.

`rdd.map(function)`

Use Cases

- Data transformation
- Feature engineering
- Column calculations

```
rdd = sc.parallelize([1,2,3,4])  
result = rdd.map(lambda x: x * 2)  
print(result.collect())
```

Output : [2,4,6,8]

2. flatMap()

Similar to map(), but flattens output.

rdd.flatMap(function)

Use Cases

- Word count
- Tokenization
- Exploding nested data



```
rdd = sc.parallelize(["hello world", "apache  
spark"])

result = rdd.flatMap(lambda x: x.split(" "))

print(result.collect())
```

Output : ['hello', 'world', 'apache', 'spark']

3. filter()

Filters elements based on condition.

rdd.filter(function)

Use Cases

- Remove invalid records
- Conditional processing
- Data cleansing



```
rdd = sc.parallelize([1,2,3,4,5,6])

result = rdd.filter(lambda x: x % 2 == 0)

print(result.collect())
```

Output : [2,4,6]

4. distinct()

Removes duplicates.

rdd.distinct()



```
rdd = sc.parallelize([1,2,2,3,3,4])

result = rdd.distinct()

print(result.collect())
```

Output : [1,2,3,4]

5. union()

Combines two RDDs.

`rdd1.union(rdd2)`



```
rdd1 = sc.parallelize([1,2,3])
rdd2 = sc.parallelize([4,5,6])

result = rdd1.union(rdd2)

print(result.collect())
```

Output : [1,2,3,4,5,6]

6. intersection()

Returns common elements.

`rdd1.intersection(rdd2)`



```
rdd1 = sc.parallelize([1,2,3,4])
rdd2 = sc.parallelize([3,4,5,6])

result = rdd1.intersection(rdd2)

print(result.collect())
```

Output : [3,4]

7. groupByKey()

Groups values by key.

`rdd.groupByKey()`



```
rdd = sc.parallelize([
    ("a",1), ("a",2), ("b",3) ])
result = rdd.groupByKey()

print(result.mapValues(list).collect())
```

Output : [('a',[1,2]), ('b',[3])]

8. reduceByKey()

Reduces values with same key.

`rdd.reduceByKey(function)`

- Performs local aggregation first
- Less shuffle
- Better performance



```
rdd = sc.parallelize([
    ("a",1), ("a",2), ("b",3)])

result = rdd.reduceByKey(lambda a,b: a+b)

print(result.collect())
```

Output : [('a',3), ('b',3)]

9. sortByKey()

Sorts pair RDD by key.

`rdd.sortByKey()`



```
rdd = sc.parallelize([
    (3,"c"), (1,"a"), (2,"b")  ])

result = rdd.sortByKey()

print(result.collect())
```

Output : [(1,'a'), (2,'b'), (3,'c')]

10. sample()

Used For Random sampling.

Parameters

- withReplacement
 - True
 - False
- fraction
 - Approximate sample size



```
rdd = sc.parallelize(range(1,11))

result = rdd.sample(False, 0.4)

print(result.collect())
```


RDD Actions

1. collect()

Returns all data to driver.

`rdd.collect()`

Best Use Cases

- Small datasets
- Debugging



```
rdd = sc.parallelize([1,2,3])  
  
print(rdd.collect())
```

Output : [1,2,3]

2. count()

Counts total number of elements.

Syntax

`rdd.count()`

Use Cases

- Dataset size checking
- Validation



```
rdd = sc.parallelize([10,20,30,40])  
  
print(rdd.count())
```

Output : 4

3. first()

Returns first element.

Syntax:

`rdd.first()`



```
rdd = sc.parallelize([100,200,300])  
  
print(rdd.first())
```

Output : 100

4. take(n)

Returns first n elements.

Syntax

rdd.take(n)



```
rdd = sc.parallelize([1,2,3,4,5])  
  
print(rdd.take(3))
```

Output : [1,2,3]

5. top(n)

Returns top n elements in descending order.

Syntax

rdd.top(n)



```
rdd = sc.parallelize([4,1,9,2,7])  
  
print(rdd.top(3))
```

Output : [9,7,4]

6. takeOrdered(n)

Returns smallest n elements.

Syntax

rdd.takeOrdered(n)



```
rdd = sc.parallelize([4,1,9,2,7])  
  
print(rdd.takeOrdered(3))
```

Output : [1,2,4]

7. reduce()

Aggregates all elements using binary function.

Syntax

`rdd.reduce(function)`



```
rdd = sc.parallelize([1,2,3,4])
result = rdd.reduce(lambda a,b: a+b)
print(result)
```

Output : [1,2,3]

8. fold()

Similar to reduce but uses initial value.

Syntax

`rdd.fold(zeroValue, function)`



```
rdd = sc.parallelize([1,2,3])

result = rdd.fold(10, lambda a,b: a+b)

print(result)
```

Output : 16

9. aggregate()

Performs complex aggregation.

Syntax

`rdd.aggregate(
 zeroValue,
 seqOp,
 combOp
)`

```
rdd = sc.parallelize([1,2,3,4])
result = rdd.aggregate(  
    0,  
    lambda acc,x: acc+x,  
    lambda acc1,acc2: acc1+acc2  
)
print(result)
```

Output : [1,2,4]

In Spark, transformations are not executed immediately.

Instead, Spark records all transformations and creates an execution plan called a DAG (Directed Acyclic Graph).

Actual execution begins only when an action is triggered.

How Lazy Evaluation Works

When transformations like `map()`, `filter()`, `flatMap()` are applied, Spark does not process the data instantly.

Instead, it:

1. Stores the transformation logic
2. Builds the DAG
3. Optimizes execution
4. Waits for an action

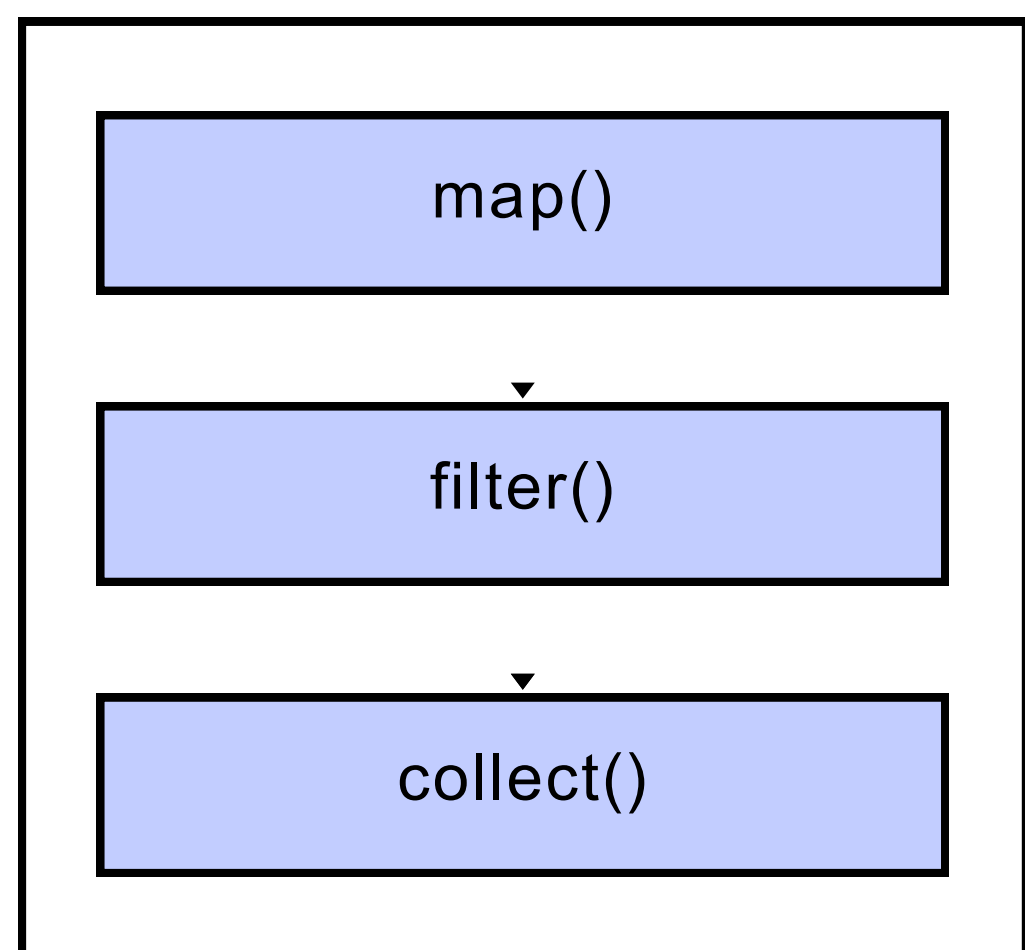


```
rdd = rdd.map(lambda x: x * 2)
rdd = rdd.filter(lambda x: x > 10)
rdd.collect()
```

Explanation

- `map()` → Transformation recorded
- `filter()` → Transformation recorded
- `collect()` → Action triggers execution

Spark executes everything only after `collect()` is called.



By default, Spark recomputes transformations every time an action is triggered. This can become expensive for:

- Iterative algorithms
- Machine Learning pipelines
- Repeated queries
- Interactive analytics

Caching stores intermediate data so Spark can reuse it directly.

The `cache()` method stores datasets in memory for faster access.

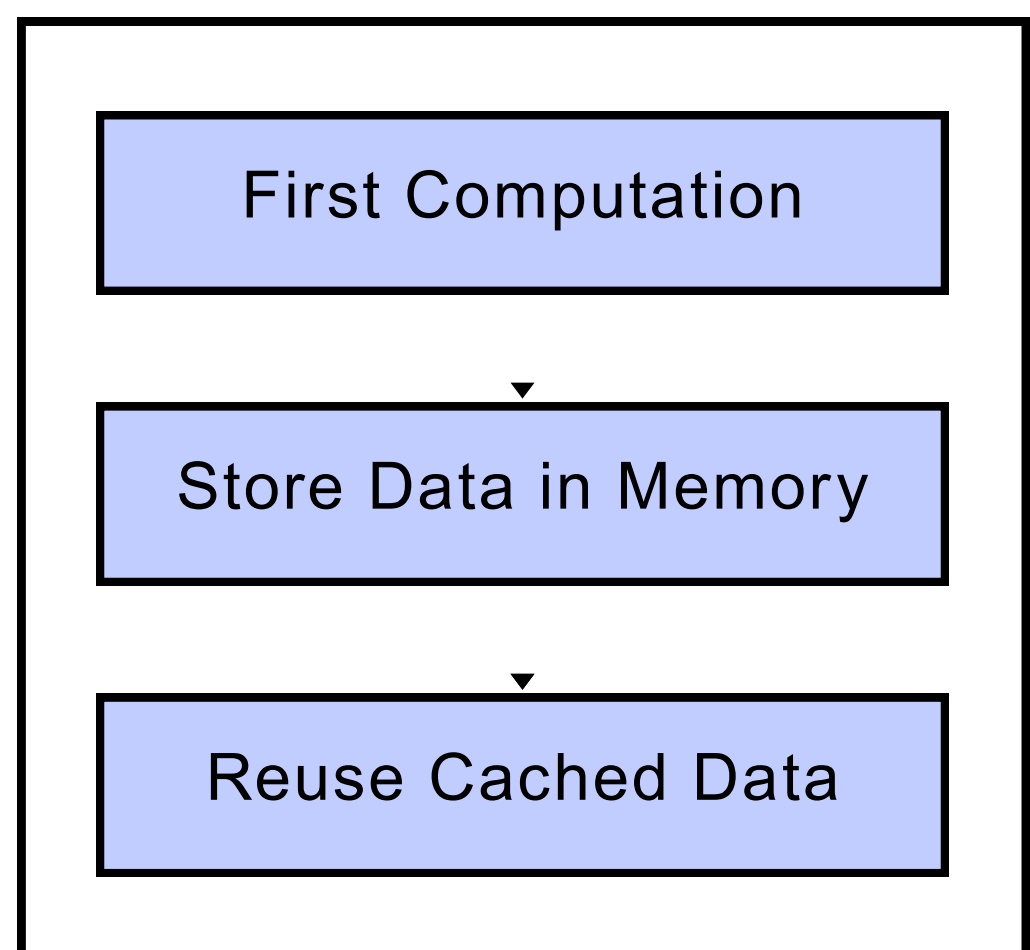
Syntax: `df.cache()`

The first execution computes and caches the data.

- Future operations reuse the cached dataset instead of recomputing it.

Advantages of `cache()`

- Faster execution
- Reduced recomputation
- Better performance for repeated operations
- Improved iterative processing

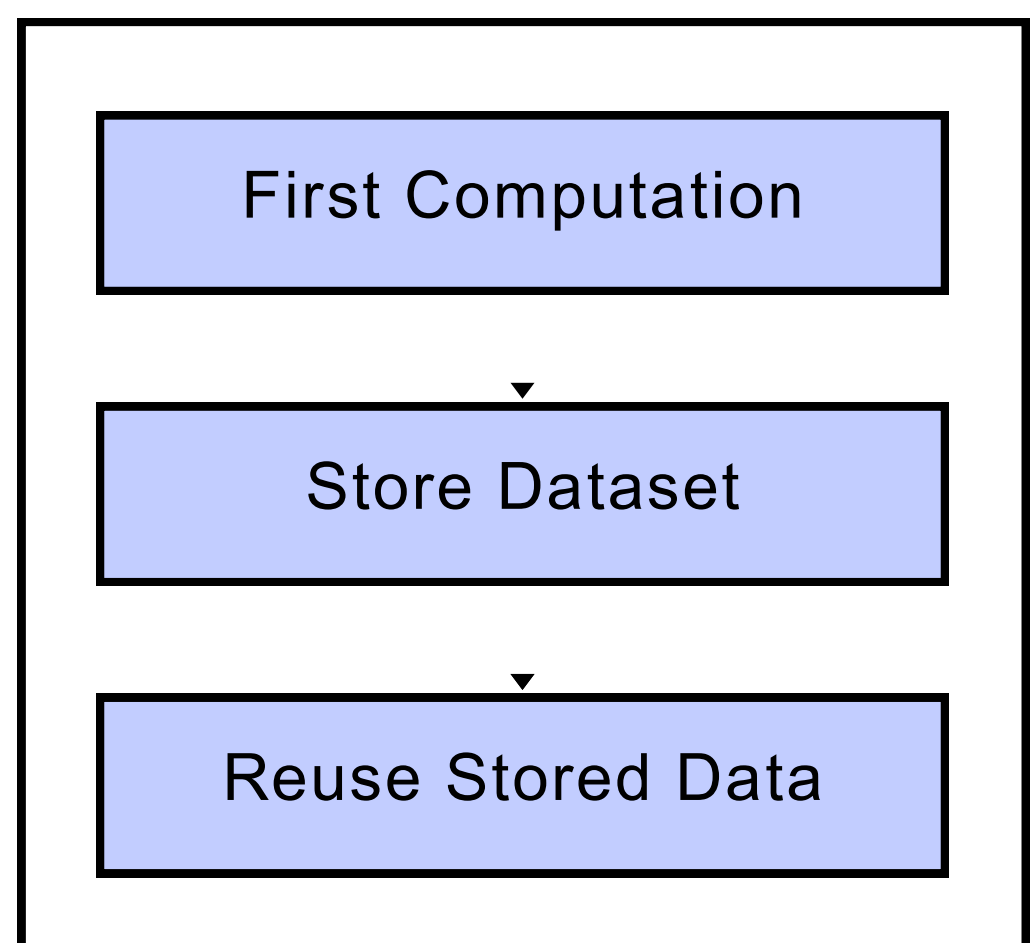


`persist()` allows Spark to store datasets using customizable storage levels.

It is useful for:

- Large datasets
- Iterative workloads
- Memory optimization
- Repeated computations

Unlike `cache()`, it can spill data to disk when memory is insufficient.



Syntax : `df.persist()`

Part IV - Introduction to Apache Hadoop

4.1

Dataframe

DataFrames are the most commonly used data abstraction in Apache Spark. A DataFrame is a distributed collection of data organized into rows and columns, similar to:

- SQL tables
- Excel sheets
- Pandas DataFrames

Unlike RDDs, DataFrames are schema-based and optimized for performance.

Features of Dataframe

1. Distributed - Data stored across cluster nodes
2. Schema-Based - Structured rows and columns
3. Optimized - Uses Catalyst Optimizer
4. Scalable - Handles large datasets efficiently
5. Easy to Use - Simple high-level APIs

Advantages of DataFrames

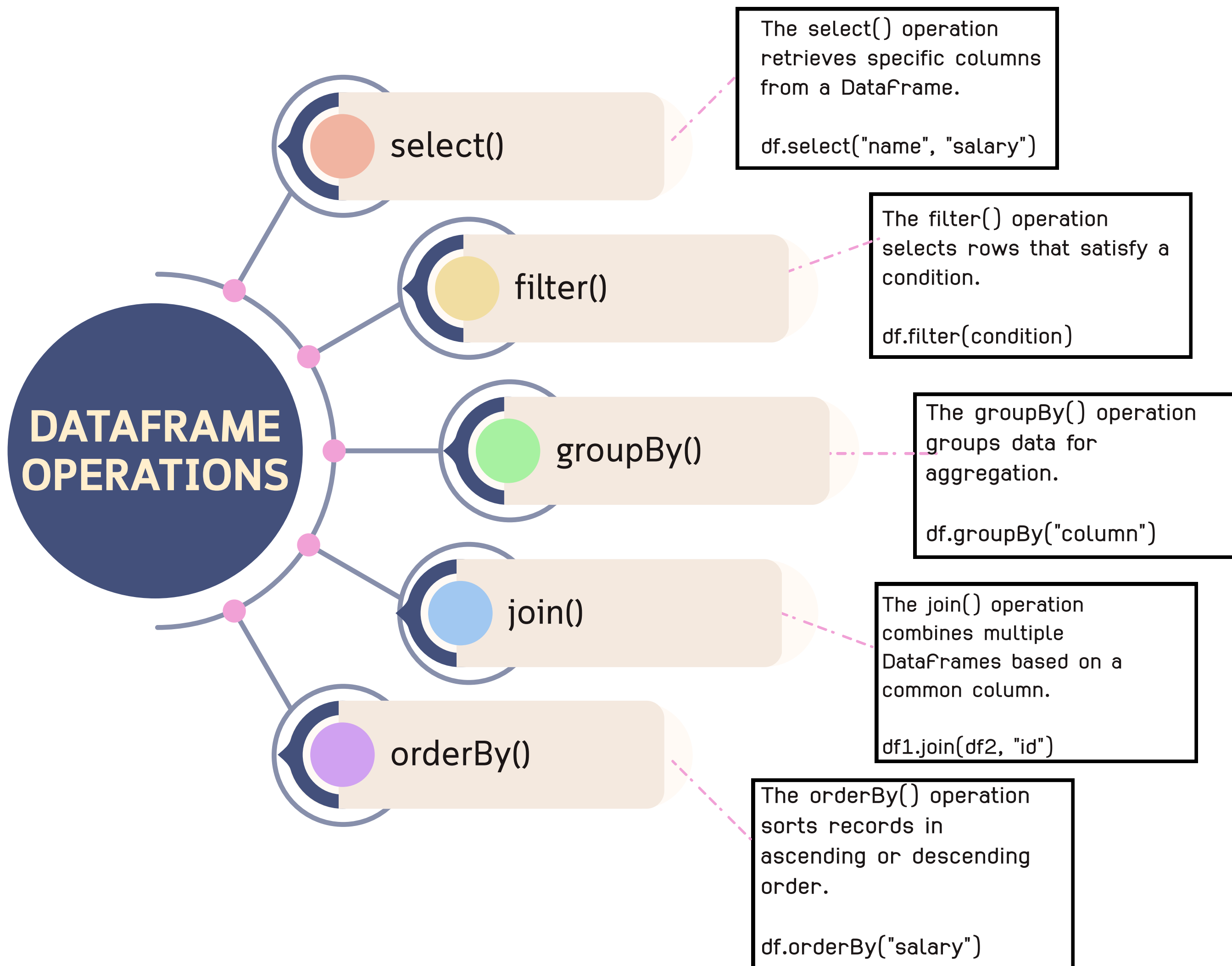
1. Better performance than RDDs
2. Easier syntax
3. Automatic query optimization
4. Efficient memory usage
5. Supports SQL queries

```
Example-  
df.spark.read.csv("employees.csv", header=True)
```

This creates a structured distributed dataset.

DataFrame

	Name	Age	Sex	label
0				row
1				
2				
3				
index	column			



Apache Spark SQL is a component of Apache Spark used for processing structured and semi-structured data using SQL queries.

It allows users to query distributed datasets using familiar SQL syntax while benefiting from Spark's distributed processing capabilities.

Features of Dataframe

1. SQL Queries - Query data using SQL syntax
2. DataFrame Support - Works with DataFrames
3. Query Optimization - Uses Catalyst Optimizer
4. Scalability - Processes large distributed datasets
5. Hive Integration - Supports Hive tables and metastore

Why Spark SQL is Important

Spark SQL combines:

- Distributed computing
- Structured data processing
- SQL querying

into a unified analytics platform.

It simplifies large-scale data analytics using familiar SQL operations.

Creating Temporary Views

A DataFrame can be registered as a temporary SQL table.

Syntax - `df.createOrReplaceTempView("employees")`

Running SQL Queries

After creating a temporary view, SQL queries can be executed using `spark.sql()`.

Example

```
spark.sql("""
SELECT department, AVG(salary)
FROM employees
GROUP BY department
""")
```

The Catalyst Optimizer is the query optimization engine used in Apache Spark SQL.

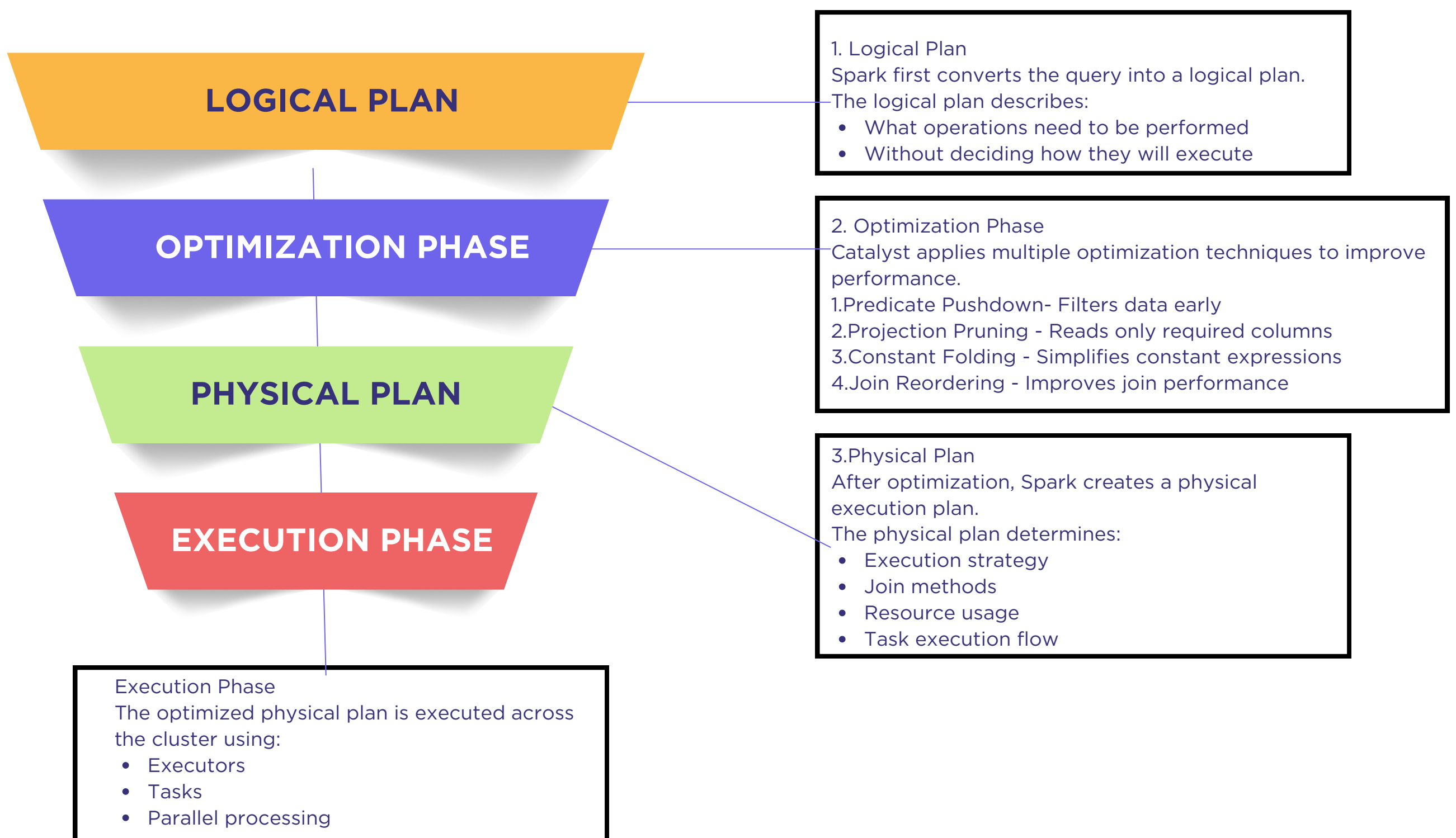
It automatically optimizes DataFrame and SQL queries to improve performance and execution efficiency in Apache Spark.

When users run SQL queries or DataFrame operations, Spark does not execute them directly.

Instead, Spark:

1. Creates a query plan
2. Optimizes the plan
3. Generates an efficient execution strategy

This process is handled by the Catalyst Optimizer.



Part V - Introduction to Apache Hadoop

5.1

Directed Acyclic Graph

A DAG (Directed Acyclic Graph) is Spark's execution plan that represents the sequence of transformations applied to data.

Spark builds the DAG before executing the application.

The DAG allows Spark to:

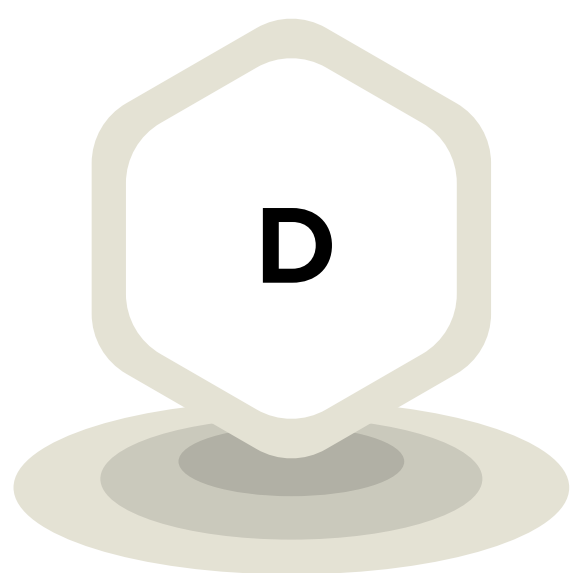
- Optimize execution
- Improve parallel processing
- Reduce unnecessary computation
- Efficiently schedule tasks

It is one of the major reasons Spark is faster than Hadoop MapReduce.

Spark transformations are not executed immediately.

Instead, Spark:

1. Records transformations
2. Builds a DAG
3. Optimizes the execution plan
4. Executes tasks when an action is triggered



DIRECTED

Operations follow a specific direction



ACYCLIC

No circular dependencies exist



GRAPH

Represents workflow as connected operations

NARROW**VS****WIDE**

In a narrow transformation, each output partition depends on only one input partition. Data remains within the same partition and does not move across nodes.

In a wide transformation, multiple input partitions contribute to a single output partition. This requires data movement across cluster nodes.

1.map()
2.filter()
3.flatMap()
4.mapPartitions()

1.groupByKey()
2.reduceByKey()
3.join()
4.distinct()

`rdd.map(lambda x: x * 2)`
Each partition is processed independently.

`rdd.groupByKey()`

Spark redistributes data based on keys.

No partition exchange occurs.

Partition Exchange Across Nodes

Minimal Disk I/O required

Higher Disk I/O required

One-to-one dependency

Many to Many Transformations

Shuffle Not required

Shuffle required

A Shuffle Operation in Apache Spark is the process of redistributing data across partitions during execution.

Shuffle occurs when Spark needs to move data between executors or worker nodes. It is one of the most expensive operations in Spark

Shuffle is required when data from multiple partitions must be reorganized or grouped together. This usually happens during:

- Aggregation
- Joining datasets
- Sorting operations

Operations That Trigger Shuffle

1. `groupBy()` - groups similar records
2. `join()` - Combines datasets
3. `reduceByKey()` - Aggregates values by key
4. `orderBy()` - Sorts distributed data

How to Reduce Shuffle Operations

1. Use Narrow Transformations

Prefer:

- `map()`
- `filter()`

when possible.

2. Reduce Unnecessary `groupBy()`

Avoid excessive aggregation operations.

3. Optimize Partitioning

Proper partitioning reduces data movement.

4. Use `reduceByKey()` Instead of `groupByKey()`

`reduceByKey()` performs partial aggregation before shuffle, reducing network overhead.

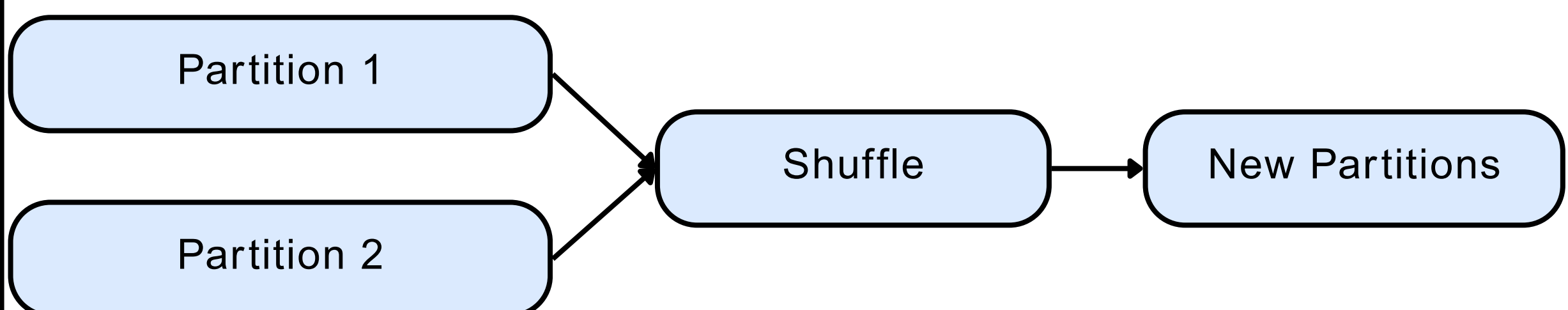
Shuffle Optimization Importance

Reducing shuffle improves:

- Spark performance
- Resource efficiency
- Query execution speed
- Scalability

Shuffle usually occurs during wide Transformations because data must move across partitions.

Example Workflow



Partitioning in Apache Spark refers to the process of dividing large datasets into smaller chunks called partitions.

Each partition is processed independently and in parallel across multiple executors in the cluster.

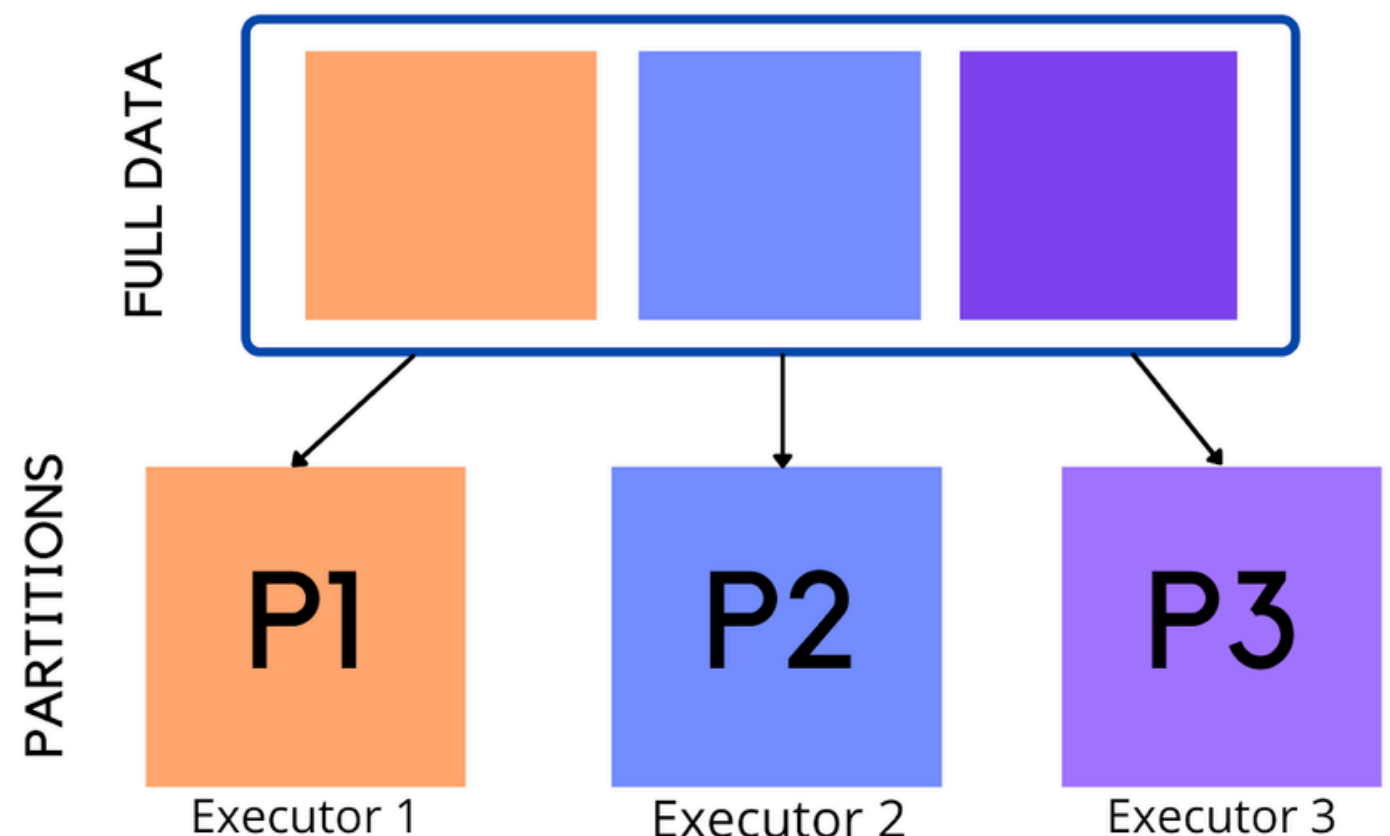
Partitioning enables Spark to:

- Process data in parallel
- Improve scalability
- Balance workloads across nodes
- Reduce execution time

Without partitioning, distributed processing would not be possible.

Example - `df.repartition(3)`

This divides the dataset into 3 partitions



Types of Partitioning

1. Hash Partitioning

Data is distributed based on a hash value of a key.

Example

$\text{hash}(\text{key}) \% \text{number_of_partitions}$

Benefits

- Uniform data distribution
- Efficient joins and aggregations

2. Range Partitioning

Data is distributed based on value ranges.

Example

Partition 1 → 1–100

Partition 2 → 101–200

Benefits

Useful for:

- Sorted data
- Range-based queries

Part VI - Spark Streaming

6.1

What is Spark Streaming

Spark Streaming is a component of Spark used for processing real-time streaming data.

It continuously ingests, processes, and analyzes incoming data from various sources.

Features of Spark Streaming

Real-Time Processing - Processes continuous incoming data

Scalable - Supports distributed streaming

Fault Tolerant - Recovers from failures automatically

Integration Support - Works with Kafka, Flume, sockets, etc.

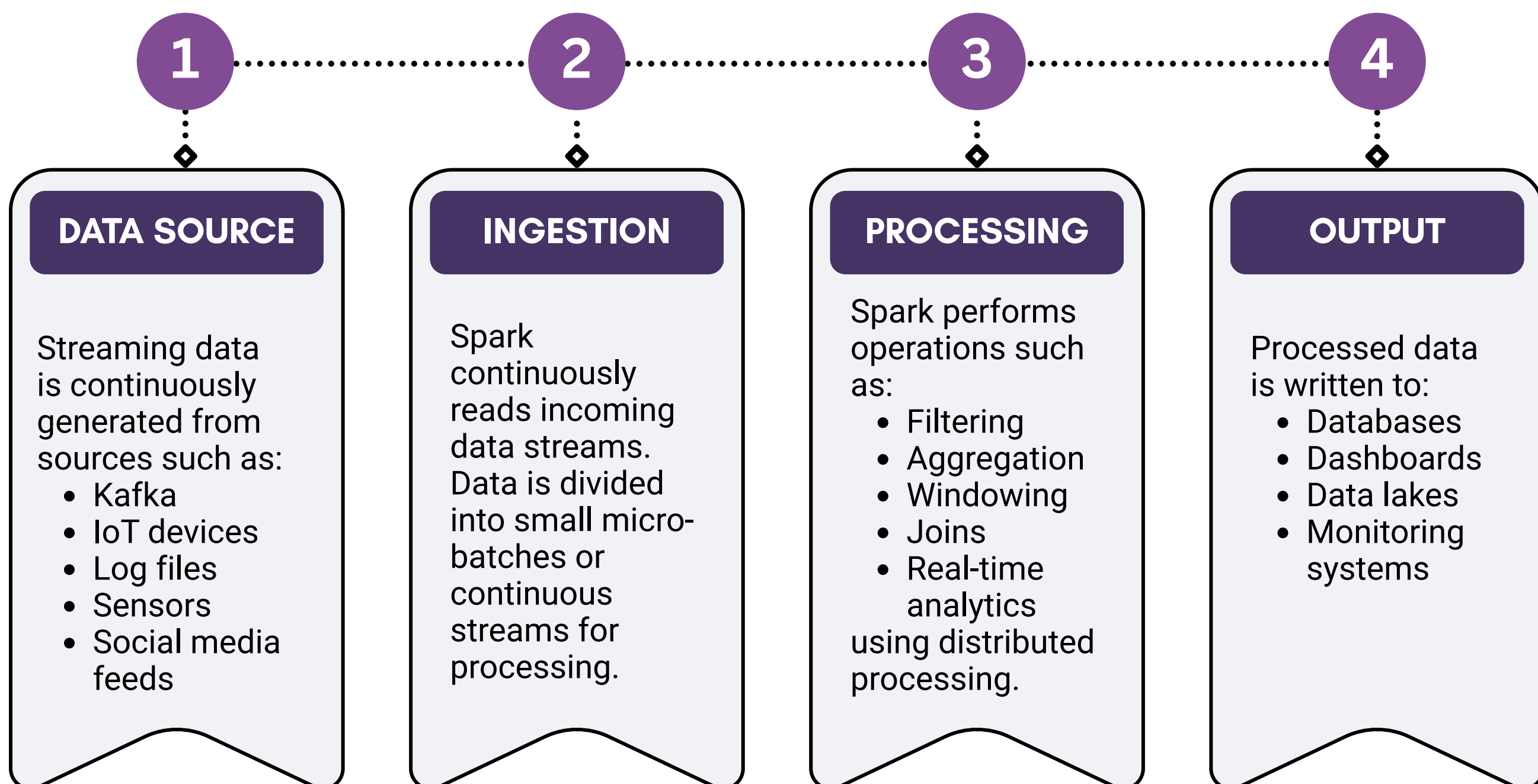
Modern Spark uses Structured Streaming instead of the older DStream

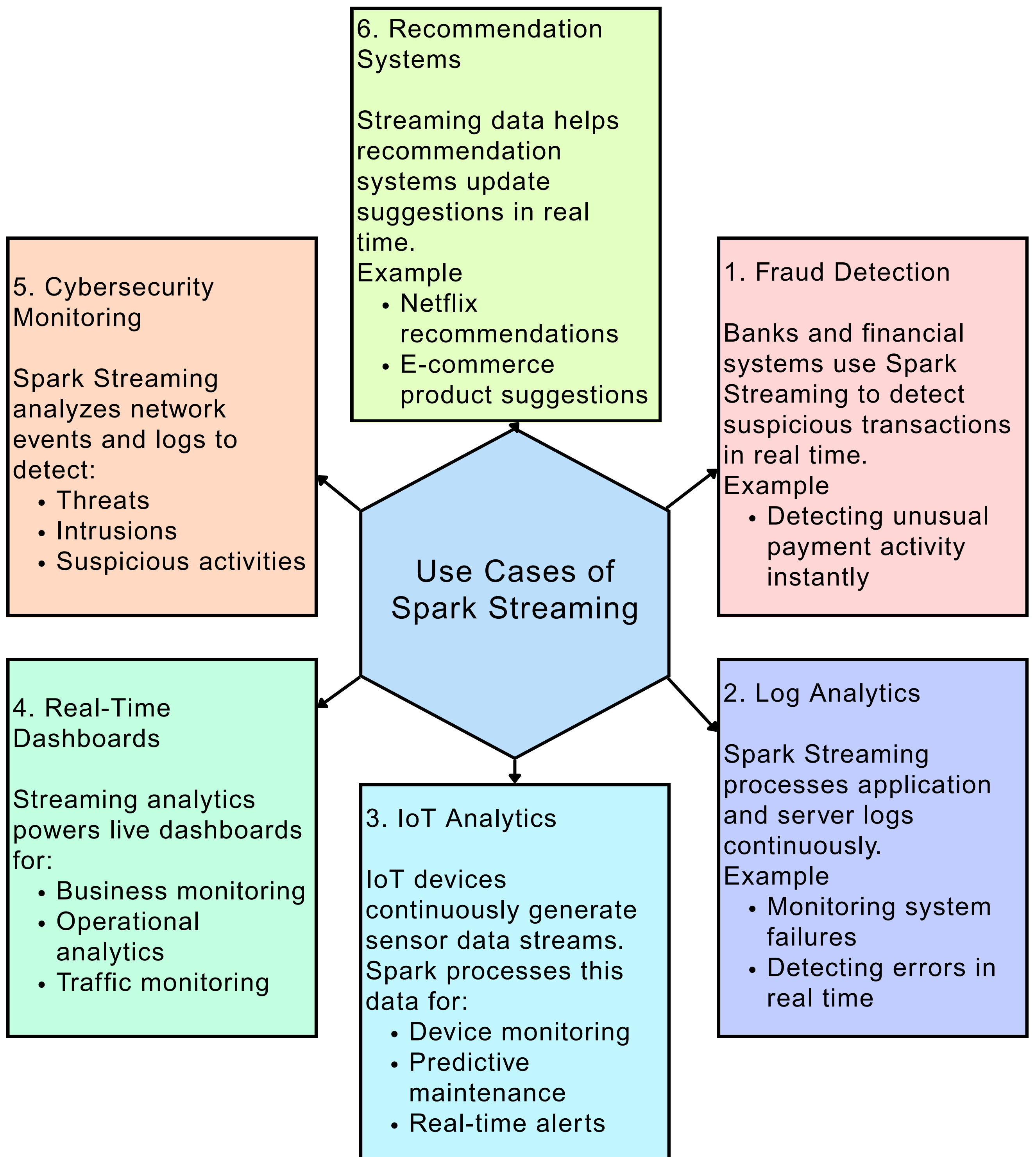
Structured Streaming processes streaming data using:

- DataFrames
- Spark SQL APIs

6.2

Streaming Workflow





Part VII - Spark Mlib

7.1

What is Mlib

MLlib is Spark's built-in Machine Learning framework used for developing scalable ML applications.

It supports:

- Data preprocessing
- Feature engineering
- Model training
- Model evaluation

using distributed computing.

Features of MLlib

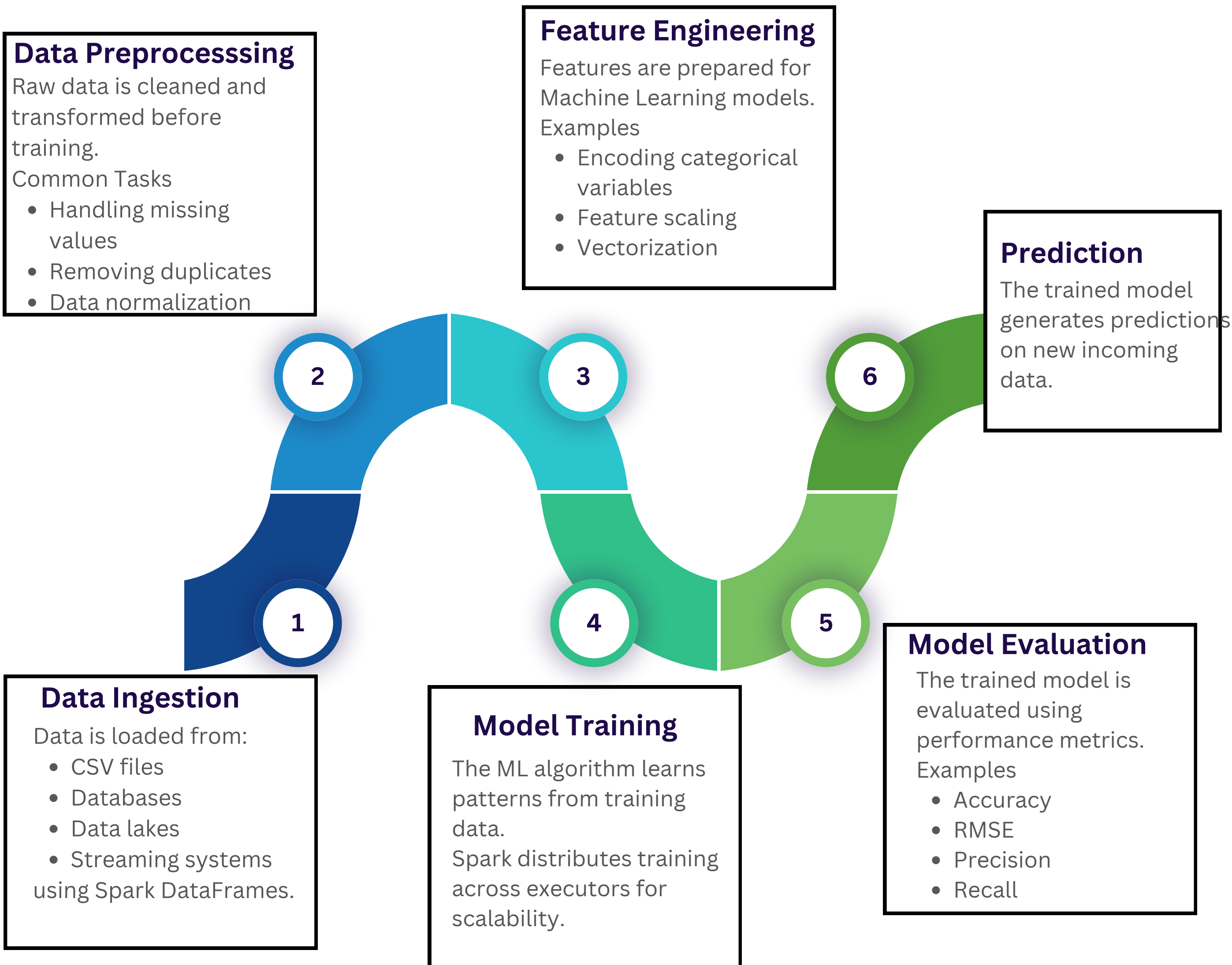
1. Distributed ML - Trains models across clusters
2. Scalable - Handles massive datasets
3. Integrated with Spark - Works with DataFrames & SQL
4. Optimized - Uses distributed parallel processing
5. Pipeline Support - End-to-end ML workflows

Traditional Machine Learning tools struggle with:

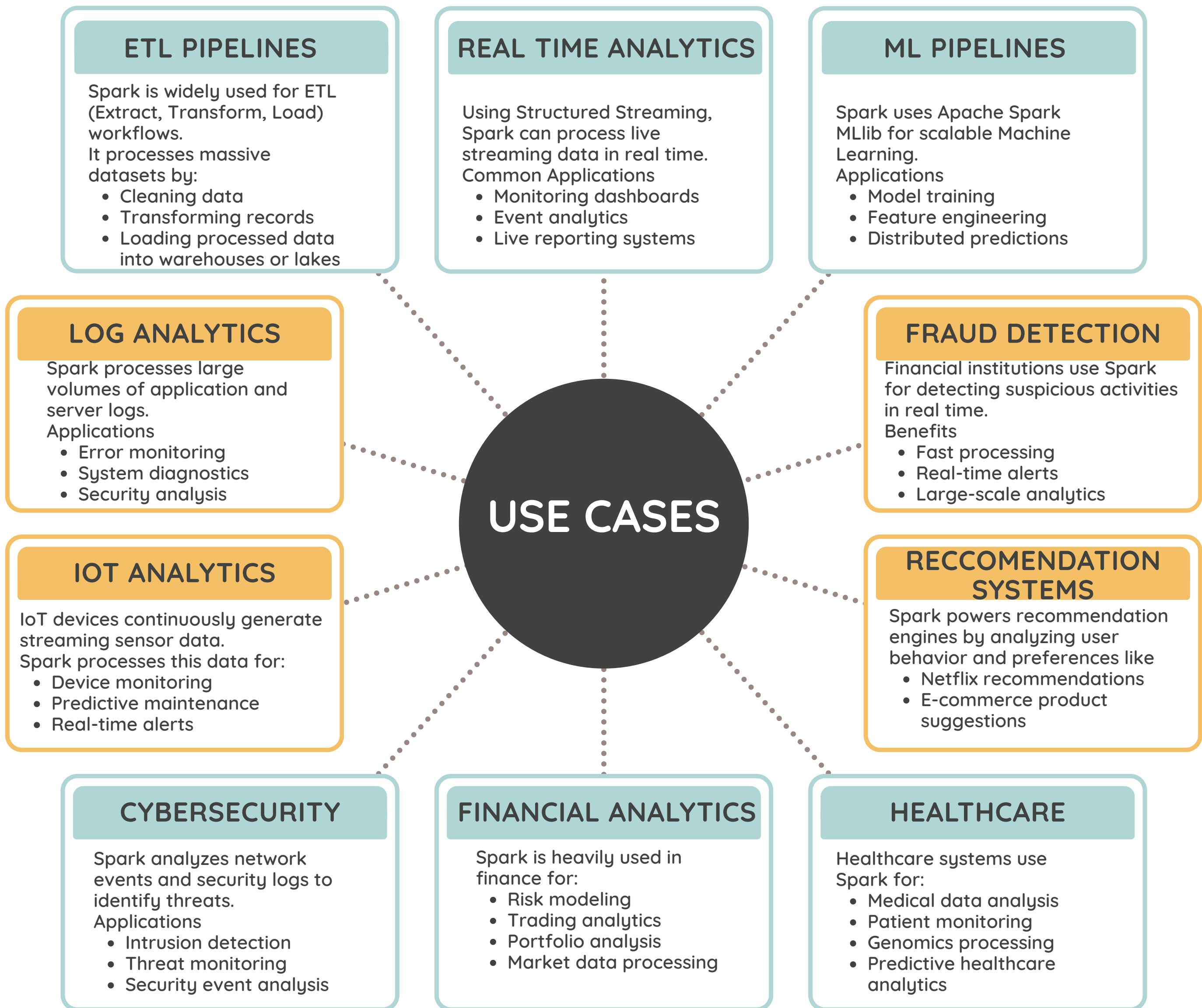
- Very large datasets
- Distributed computation
- Scalability

MLlib solves these problems using:

- Parallel processing
- In-memory computation
- Distributed execution



Part VIII - Spark Use Cases



Part IX - Spark Installation

1

Update Linux Packages

```
sudo apt update && sudo apt upgrade -y
```

2

Install Java (Required)

```
sudo apt install openjdk-23-jdk -y  
java -version
```

3

Install Python

```
sudo apt install python3 python3-pip -y  
python3 -version
```

3

Install Spark

```
wget https://downloads.apache.org/spark/spark-3.5.6/spark-3.5.6-bin-hadoop3.tgz  
  
tar -xvzf spark-3.5.6-bin-hadoop3.tgz  
  
sudo mv spark-3.5.6-bin-hadoop3 /opt/spark
```

Open .bashrc

```
nano ~/.bashrc
```

Add these lines at the bottom:

```
# JAVA
export JAVA_HOME=/usr/lib/jvm/java-23-openjdk-amd64

# SPARK
export SPARK_HOME=/opt/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin

# PYSPARK
export PYSARK_PYTHON=python3
```

Reload Bash

```
source ~/.bashrc
```

```
spark-shell
```



Spark Integration with Tools

1

Hadoop Integration

Set Environment Variables

Open .bashrc

```
nano ~/.bashrc
```

Add these lines at the bottom:

```
# HADOOP
export HADOOP_HOME=/opt/hadoop
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop

# SPARK
export SPARK_HOME=/opt/spark

# PATH
export PATH=$PATH:$HADOOP_HOME/bin:$SPARK_HOME/bin
```

Reload Bash

```
source ~/.bashrc
```

Verify Hadoop

```
start-dfs.sh
jps
```

Expected

```
NameNode
DataNode
SecondaryNameNode
```

Verify YARN

```
start-yarn.sh  
jps
```

Expected

```
ResourceManager  
NodeManager
```

Test HDFS

Create HDFS Directory

```
hdfs dfs -mkdir /spark-demo
```

Upload File

```
hdfs dfs -put employees.csv /spark-demo
```

Verify

```
hdfs dfs -ls /spark-demo
```

Create SparkSession with hadoop

```
from pyspark.sql import SparkSession  
  
spark = SparkSession.builder \  
    .appName("SparkHadoopIntegration") \  
    .master("yarn") \  
    .getOrCreate()
```


Read Data from HDFS

```
df = spark.read.csv(
    "hdfs://localhost:9000/spark-demo/employees.csv",
    header=True,
    inferSchema=True
)

df.show()
```

Write Data to HDFS

```
df.write.mode("overwrite").csv(
    "hdfs://localhost:9000/output/employees"
)
```

Read Text File from HDFS using RDD

```
rdd = spark.sparkContext.textFile(
    "hdfs://localhost:9000/spark-demo/data.txt"
)

print(rdd.collect())
```

Word Count Example

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("WordCount") \
    .master("yarn") \
    .getOrCreate()

rdd = spark.sparkContext.textFile(
    "hdfs://localhost:9000/input/data.txt"
)
word_counts = rdd.flatMap(
    lambda line: line.split(" ")) \
    .map(
        lambda word: (word, 1)) \
    .reduceByKey(
        lambda a,b: a+b
    )
word_counts.saveAsTextFile(
    "hdfs://localhost:9000/output/wordcount"
)
spark.stop()
```

Submit Spark Job on YARN

```
spark-submit \
--master yarn \
--deploy-mode client \
wordcount.py
```

Submit Spark Job on YARN

```
spark-submit \
--master yarn \
--deploy-mode cluster \
wordcount.py
```

Start Zookeeper

```
zookeeper-server-start.sh \
config/zookeeper.properties
```

Start Kafka Server

```
kafka-server-start.sh \
config/server.properties
```

Create Kafka Topic

```
kafka-topics.sh \
--create \
--topic spark-demo \
--bootstrap-server localhost:9092 \
--partitions 1 \
--replication-factor 1
```

Verify Topic

```
kafka-topics.sh \
--list \
--bootstrap-server localhost:9092
```

Start Kafka Producer

```
kafka-console-producer.sh \  
--topic spark-demo \  
--bootstrap-server localhost:9092
```

Spark Kafka Package

```
spark-submit \  
--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1
```

Create SparkSession

```
from pyspark.sql import SparkSession  
  
spark = SparkSession.builder \  
    .appName("SparkKafka") \  
    .getOrCreate()
```

Read Stream from Kafka

```
df = spark.readStream \  
    .format("kafka") \  
    .option("kafka.bootstrap.servers", "localhost:9092") \  
    .option("subscribe", "spark-demo") \  
    .load()
```


Word Count Streaming Example

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode, split

spark = SparkSession.builder \
    .appName("KafkaWordCount") \
    .getOrCreate()

df = spark.readStream \
    .format("kafka") \
    .option(
        "kafka.bootstrap.servers",
        "localhost:9092"
    ) \
    .option("subscribe", "spark-demo") \
    .load()

lines = df.selectExpr(
    "CAST(value AS STRING)"
)

words = lines.select(
    explode(
        split(lines.value, " ")
    ).alias("word")
)

word_counts = words.groupBy("word").count()

query = word_counts.writeStream \
    .outputMode("complete") \
    .format("console") \
    .start()

query.awaitTermination()
```

Install Airflow and Spark Provider

```
pip install apache-airflow apache-airflow-providers-apache-spark
```

Initialize Airflow Database

```
airflow db init
```

Create Airflow User

```
airflow users create \  
--username admin \  
--firstname admin \  
--lastname admin \  
--role Admin \  
--email admin@example.com
```

Start Airflow

```
airflow webserver --port 8080
```

Basic Spark Job and Airflow DAG

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("AirflowSparkJob") \
    .getOrCreate()
data = [
    ("Alice", 50000),
    ("Bob", 60000)
]
df = spark.createDataFrame(
    data,
    ["name", "salary"])

df.show()
spark.stop()
```

```
from airflow import DAG
from airflow.providers.apache.spark.operators.spark_submit
import SparkSubmitOperator
from datetime import datetime

with DAG(
    dag_id="spark_job_dag",
    start_date=datetime(2025, 1, 1),
    schedule_interval="@daily",
    catchup=False
) as dag:

    spark_task = SparkSubmitOperator(
        task_id="run_spark_job",

        application="/home/user/spark_job.py",

        conn_id="spark_default",

        verbose=True
    )
```

Hive Setup

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("SparkHive") \
    .enableHiveSupport() \
    .getOrCreate()
```

```
spark.sql("SHOW DATABASES").show()
```

Hive Database and Table Setup

```
spark.sql("""
CREATE DATABASE IF NOT EXISTS company
""")
```

```
spark.sql("""
USE company
""")
```

```
spark.sql("""
CREATE TABLE employees (
    id INT,
    name STRING,
    salary DOUBLE
)
STORED AS PARQUET
""")
```